Лабораторна робота №3

Тема: Одновимірні та багатовимірні масиви, обробка символьних рядків у мові С та робота з гілками в Git

Посилання на GitHub

Мета роботи

Набути практичних навичок обробки послідовних областей оперативної пам'яті: проектування алгоритмів фільтрації, пошуку екстремумів та перестановок елементів в одновимірних масивах.
Опанувати кеш-дружні алгоритми обробки двовимірних матриць з урахуванням розташування даних за рядками (Row-Major Order).
Засвоїти низькорівневу специфіку роботи з текстовою інформацією через символьні масиви типу char із кінцевим нуль-термінатором (\0).
Опанувати командну модель розгалуження в системі контролю версій Git: створення функціональних гілок, фіксацію змін та їхнє безпечне злиття в основну гілку.
Підготувати підсумкову інженерну документацію у форматі PDF.

Етап 1. Обробка одновимірного масиву (task1.c)
Знайти суму додатних елементів масиву та змінити знак усіх елементів, розташованих до першого мінімального.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.F7.5.000 –Лр.3		
Змн.	*Арк.*	*№ докум.*	*Підпис*	*Дат*			
Розроб.		*Ничипорчук І.В*			*Звіт з лабораторної роботи №3*	*Літ.*	*Арк.* 8 / *Акрушів* 8
Перевір.		*Лисенко М.С*					
Реценз.						*ФІКТ, гр. КІ-26-1*	
Н. Контр.							
Зав.каф.		Лобанчикова Н.М					

Код програми task1.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 15

int main(void) {
    const int MIN_VALUE = -50;
    const int MAX_VALUE = 50;
    int arr[SIZE];

    // Заповнення масиву псевдовипадковими числами з [-50; 50]
    srand((unsigned int)time(NULL));
    for (int i = 0; i < SIZE; i++) {
        arr[i] = rand() % (MAX_VALUE - MIN_VALUE + 1) + MIN_VALUE;
    }

    printf("Original array:\n");
    for (int i = 0; i < SIZE; i++) {
        printf("%4d", arr[i]);
    }
    printf("\n");

    // Еталон для мінімуму: нульовий елемент
    int min_value = arr[0];
    int min_index = 0;
    int sum_positive = 0;

    for (int i = 0; i < SIZE; i++) {
        // Сума додатних елементів
        if (arr[i] > 0) {
            sum_positive += arr[i];
        }
        // Строге порівняння зберігає ПЕРШИЙ мінімальний елемент
        if (arr[i] < min_value) {
            min_value = arr[i];
            min_index = i;
        }
    }
```

```c
    // Зміна знаку всіх елементів до першого мінімального
    for (int i = 0; i < min_index; i++) {
        arr[i] = -arr[i];
    }

    printf("\nSum of positive elements: %d\n", sum_positive);
    printf("First minimum: %d at index %d\n", min_value, min_index);
    printf("Modified array:\n");
    for (int i = 0; i < SIZE; i++) {
        printf("%4d", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Компіляція виконувалася командою:gcc -Wall -Wextra task1.c -o task1

.\task1.exe

Результат виконання

```
PS C:\Users\illia\projects\programming-sem1\lab03> .\task1.exe
Original array:
  35 -29 -26  28 -47 -32 -29  31 -42  41  24  -1 -46  30 -48

Sum of positive elements: 189
First minimum: -48 at index 14
Modified array:
 -35  29  26 -28  47  32  29 -31  42 -41 -24   1  46 -30 -48
```

Програма оголошує статичний масив із 15 цілих чисел, розмір якого задано макросом SIZE. Масив заповнюється псевдовипадковими числами з діапазону [–50; 50] за допомогою функцій srand() та rand() із бібліотек stdlib.h і time.h. Межі діапазону оголошено константами MIN_VALUE та MAX_VALUE, щоб не використовувати «магічні числа». Початковий масив виводиться в один рядок. Далі еталонну змінну min_value ініціалізовано значенням нульового елемента масиву. У циклі for одночасно виконуються дві дії: до суми sum_positive додаються додатні елементи, а якщо поточний елемент менший за min_value, оновлюються мінімум і його індекс. Строге порівняння (<) гарантує, що буде знайдено саме перший мінімальний елемент. Другий цикл змінює знак усіх елементів з індексами від 0 до min_index – 1. Сам мінімальний елемент не змінюється. Наприкінці виводяться сума додатних елементів, перший мінімум з його індексом та перетворений масив.

Результат: сума додатних елементів дорівнює 189, перший мінімум –48 знаходиться на індексі 14, усі елементи перед ним змінили знак.

Етап 2. Кеш-дружня обробка 2D-матриці (task2.c)
Порахувати кількість додатних чисел у кожному рядку матриці та зберегти їх в окремий масив-вектор**.**

Код програми task2.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define ROWS 4
#define COLS 5

int main(void) {
    const int MIN_VALUE = -9;
    const int MAX_VALUE = 9;
    int matrix[ROWS][COLS];
    int positive_count[ROWS] = {0};  // вектор результатів

    srand((unsigned int)time(NULL));

    // Row-Major Order: зовнішній цикл по рядках, внутрішній по стовпцях
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            matrix[i][j] = rand() % (MAX_VALUE - MIN_VALUE + 1) + MIN_VALUE;
        }
    }

    printf("Matrix:\n");
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%6d", matrix[i][j]);
        }
        printf("\n");
    }

    // Підрахунок додатних елементів у кожному рядку
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            if (matrix[i][j] > 0) {
                positive_count[i]++;
            }
        }
    }
```

```c

    printf("\nPositive elements per row:\n");
    for (int i = 0; i < ROWS; i++) {
        printf("Row %d: %d\n", i + 1, positive_count[i]);
    }

    return 0;
}
```

Компіляція виконувалася командою:gcc -Wall -Wextra task2.c -o task2
.\task2.exe

Результат виконання

```
PS C:\Users\illia\projects\programming-sem1\lab03> .\task2.exe
Matrix:
     3     7     6    -9     9
     1     7     6    -2    -4
     1    -1     7     5     2
     0     9    -7    -5    -4

Positive elements per row:
Row 1: 4
Row 2: 3
Row 3: 4
Row 4: 1
```

Програма оголошує матрицю розміром 4 × 5, розміри якої задано макросами ROWS і COLS, та вектор positive_count для збереження результатів. Матриця заповнюється псевдовипадковими цілими числами з діапазону [−9; 9] за допомогою функцій srand() та rand(). Межі діапазону оголошено константами MIN_VALUE і MAX_VALUE, щоб не використовувати «магічні числа».

Усі вкладені цикли організовано за принципом Row-Major Order: зовнішній цикл перебирає індекс рядка, внутрішній індекс стовпця. У мові С двовимірний масив зберігається в пам'яті рядок за рядком, тому така послідовність забезпечує лінійне проходження пам'яті та ефективну роботу кеш-ліній процесора.

Для кожного рядка лічильник positive_count[i] збільшується на одиницю, якщо елемент більший за нуль. Нуль додатним числом не вважається. Матриця в цьому варіанті не змінюється, результатом обробки є вектор. Наприкінці програма виводить матрицю та

кількість додатних елементів у кожному рядку.
Результат: для показаної матриці у вектор записано значення 4, 3, 4 і 1

Етап 3. Ручна низькорівнева обробка С-рядка (task3.c)
Замінити всі послідовності з кількох пробілів, що йдуть підряд, на один одиночний пробіл. Алгоритм реалізувати вручну без функцій бібліотеки string.h.

Код програми task3.c

```c
#include <stdio.h>

#define MAX_LEN 100
#define BUFFER_SIZE (MAX_LEN + 1)  // правило N + 1 байт під '\0'

int main(void) {
    char str[BUFFER_SIZE];

    printf("Enter text (max %d chars): ", MAX_LEN);
    if (fgets(str, sizeof(str), stdin) == NULL) {
        printf("Error: input failed.\n");
        return 1;
    }

    // Ручне покрокове сканування: read - індекс читання, write - індекс запису
    int read = 0;
    int write = 0;
    int prev_space = 0;  // чи був попередній символ пробілом

    while (str[read] != '\0' && str[read] != '\n') {
        if (str[read] == ' ') {
            // Записуємо лише перший пробіл із послідовності
            if (!prev_space) {
                str[write++] = ' ';
            }
            prev_space = 1;
        } else {
            str[write++] = str[read];
            prev_space = 0;
        }
        read++;
    }
    str[write] = '\0';  // завершальний нуль-термінатор

    printf("Result: \"%s\"\n", str);

    return 0;
}
```

Компіляція виконувалася командою: gcc -Wall -Wextra task3.c -o task3
.\task3.exe

Результат виконання

```
PS C:\Users\illia\projects\programming-sem1\lab03> .\task3.exe
Enter text (max 100 chars): hello     world   and    spaces
Result: "hello world and spaces"
```

Програма оголошує символьний буфер str розміром BUFFER_SIZE, який дорівнює MAX_LEN + 1. Один додатковий байт відведено під завершальний нуль-термінатор '\0' за правилом «N + 1 байт». Рядок зчитується безпечною функцією fgets(str, sizeof(str), stdin), яка не допускає виходу за межі буфера і запобігає його переповненню на стеку. Повернене значення fgets перевіряється: якщо введення не вдалося, програма виводить повідомлення про помилку.
Обробка виконана вручну, без функцій бібліотеки string.h. Рядок сканується покроково в циклі while до символу '\0' або символу нового рядка '\n' за допомогою двох індексів: read (позиція читання) і write (позиція запису). Прапорець prev_space показує, чи був попередній символ пробілом. Пробіл записується лише тоді, коли попередній символ пробілом не був, тому з послідовності пробілів лишається один. Усі інші символи копіюються без змін.
Після завершення циклу в позицію write записується '\0', тому змінений рядок коректно завершується нуль-термінатором. Усі зсуви виконуються на місці, у тому самому масиві.
Результат: рядок «hello world and spaces» перетворився на «hello world and spaces».

Висновок

Під час виконання лабораторної роботи я опрацював одновимірні та двовимірні масиви й символьні рядки мови С.

На етапі 1 заповнив одновимірний масив псевдовипадковими числами з діапазону [−50; 50] за допомогою srand() і rand(), знайшов суму додатних елементів та змінив знак усіх елементів до першого мінімального. Еталонну змінну для пошуку мінімуму ініціалізував нульовим елементом масиву, а строге порівняння гарантувало знаходження саме першого мінімуму.

На етапі 2 обробив двовимірну матрицю 4 × 5 за принципом Row-Major Order: зовнішній цикл перебирає рядки, внутрішній стовпці, що забезпечує лінійне проходження пам'яті та ефективну роботу кеш-ліній процесора. Кількість додатних елементів кожного рядка зберіг в окремий вектор.

На етапі 3 вручну, без функцій бібліотеки string.h, реалізував заміну послідовностей пробілів на один пробіл. Рядок зчитував безпечною функцією fgets, розмір буфера визначив за правилом N + 1 байт, а після обробки коректно записав завершальний нуль-термінатор.

Також опанував командну модель розгалуження в Git: створив окрему гілку lab03, зафіксував роботу атомарними комітами за стандартом Conventional Commits і злив гілку в main командою git merge.